

Brightway version 2.5 Cheatsheet for Beginners

Numbering = consecutive actions

a., b., = different ways of how to do this

> Everything around BW

Get a minimum understanding of the following:

- **Anaconda/Miniconda:** The engine to run the BW.
- **Python:** The language in which BW is written.
- **Jupyter:** Used to create and share notebooks which contain your LCA study code
- **Git(hub):** Sharing code openly, collaborating
- **Activity Browser:** Graphical User Interface of BW
- **BW:** docs.brightway.dev // learn.brightway.dev // live.brightway.dev // try.brightway.dev
- **I need help:** check/post on brightway.groups.io (subgroups)
- **bw ecosystem** of open packages like premise, timex etc.: <https://premisedash-6f5a0259c487.herokuapp.com/ecosystem/>

> Installing, opening, upgrading bw – in prompt window

Open your (ana-/mini-)conda prompt window – this is always the start of your work in bw!

«Installing» = creating a conda environment and attaching the bw package to it:

```
conda create -n yourenv -c conda-forge -c cmutel brightway25
```

Starting = opening bw: we want to open a jupyter notebook in the environment you want to work with:

open anaconda prompt

```
(conda env list for knowing which envs you have) conda activate yourenv jupyter lab or jupyter notebook)
```

Updating: conda activate yourenv

```
conda update -c conda-forge -c cmutel brightway25
```

> Starting to use bw – in jupyter notebook

1. «open bw» = open your jupyter notebook (see above)
2. Import all necessary packages:

Minimum is this:

```
import bw2io as bi // import bw2data as bd //import bw2calc as bc
```

Often, on top: #data science

```
import numpy as np //import pandas as pd
```

```
#plotting import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

> Working on projects – in jupyter notebook

bd.projects. press tab → shows you all functions for project handling. Choose one of them, and add ? → shows you how this function works

list(bd.projects) (which projects do I have)

bd.projects.dir (where does my current project sit)

bd.projects.report() (general information)

bd.projects.current (checking in which project you are)

Activate your project/create one

a. bd.projects.set_current('myproject') (activates myproject, and creates it first if it doesn't exist yet)

b. bd.projects.create_project('myproject') (creates project, but you remain in your current project)

b. **Rename/Copy/Delete a project**

bd.projects.rename_project('newname')

bd.projects.copy_project('newname', switch=True)

(copies the current project, and if you add the «switch=True» it switches to this new copy.)

bd.projects.delete_project('myproject', delete_dir=True) (deletes myproject, and also deletes it from your computer directory)

> Import databases – biosphere, methods, ecoinvent

bd.databases (which databases do I have in my project)

a. biosphere, ecoinvent

```
bi.import_ecoinvent_release('versionnumber', 'systemmodel', 'ecoinvent User Name', 'ecoinvent password')
```

System model = cutoff / apos / consequential / EN15804

b1. biosphere

bi.bw2setup() (creates the 'biosphere3' db and imports the LCIA methods)

b2. Any other database with spold2 format, e.g. ecoinvent

```
fp = r'C:\ecoinvent_3.11_cutoff_ecoSpold02\datasets' (filepath example)
```

```
ei311imp = bi.SingleOutputEcospold2Importer(fp, 'ev311cutoff') (give your own name to the db)
```

```
ei311imp.apply_strategies()
```

```
ei311imp.statistics()
```

```
if len(list(ei311imp.unlinked)) == 0:
```

```
ei311imp.write_database()
```

> Import databases – own data

Your own LCI from an excel sheet

```
imp = bi.ExcelImporter(r'C:\yourpath')
imp.apply_strategies()
imp.match_database('ev311cutoff', fields=('name', 'reference product', 'unit', 'location'))
imp.statistics()
```

Which are the unlinked flows? → list(imp.unlinked) or

```
imp.write_excel() # (only_unlinked=True)
```

Normally, you MUST find the error leading to the unlinked flow.

Usually it's typos, wrong geographies/databases etc.

If you KNOW that you don't need it, you can do this:

```
imp.drop_unlinked(i_am_reckless=True)
```

In the end, write the database

```
imp.write_database()
```

→ It is good practice to do few quick checks to see if the db is functional: length, do LCA calc (see next page of cheatsheet for this)

SimaPro/openLCA datasets – not painless yet but soon easier

SP:<https://github.com/romainsacchi/brightpath>

oLCA:<https://github.com/cyrillefrancois/openlca2bw>

> Manage databases

Checking which dbs you have: list(bd.databases)

Delete: del bd.databases['dbname']

Copy: original = bd.Database('mydb')

```
copy = original.copy('mydb_newname')
```

Rename: original.rename('mydb_renamed')

> Activity Browser – installation & opening

Create projects, import databases, look at the databases, create own datasets, do LCIA calculations – and many helpful things more!

Recommendation: Set up your own LCI in a spreadsheet

(reproducibility, e.g. linking to original data&calculations) and

import it. If database imports fails in AB, it may be easier to do the

import via a jupyter notebook, because you can identify unlinked

exchanges. But once you have the database(s), it may be more

human friendly to look at it in AB and do quick LCIA calcs there.

When you create an env in the prompt window, set up a project&import databases in jupyter notebook, all these will directly be displayed in AB (and vice versa)!

Installation (in prompt window) = creating a conda env and

attaching the ab package to it: <https://lca-activitybrowser.github.io/activity-browser/beta.html>

Opening AB: conda activate ab

activity-browser

> Checking the content of the dbs

```
bio = bd.Database('biosphere3')
ei = bd.Database('ev311cutoff')
m = bd.methods
```

Misc, applicable for all above (except sometimes m, i.e. replace as you want with bio, ei or m):

```
len(bio) // type(bio)
act=ei.random() #picking a random activity/biosphere flow/method
act.as_dict() #Looking at an activity/biosphere flow
Diving deeper: looking at all technosphere/biosphere exchanges of an activity:
```

```
a. list(act.technosphere()) or
list(act.biosphere()) or list(act.production())
b. for e in act.exchanges():
    if e['type'] == 'technosphere':
        print(e['name'], e['amount'], e['type'],
              e.get('location'))
```

Searching:

Knowing for which fields I can search (categories):

in *ecoinvent*: `act = ei.random()` then `list(act.keys())`
In *biosphere*: `set(list(f['categories'] for f in bio))`

a. Search function:

```
ei.search('electricity')
bio.search('carbon dioxide', filter =
{'categories': 'urban', 'name': 'fossil'})
```

b. List comprehension, examples for each database:

```
Biosphere: co2 = [ flow for flow in bio
    if 'Carbon dioxide' in flow['name']
    and 'fossil' not in flow['name']
    and flow['categories'] == ('soil',) ]
```

```
Ecoinvent: coalDE = [a for a in ei
    if 'electricity production' in a['name']
    and 'coal' in a['name']
    and a['location'] == 'DE' ]
```

Methods: `ef31 = [m for m in bd.methods if 'EF v3.1' in str(m) and 'no LT' not in str(m)]`

→ Once you have that list, you can choose a list element (=specific LCIA method) with e.g. `[1]`. Then, you can explore that element, e.g. `bd.Method(ef31[0]).metadata`

```
bd.Method(ef31[0]).metadata['unit']
```

You can also use the full list for LCIA calculations.

> Calculating LCIA results for 1 activity&1 method

1. Choose an activity and give it a variable name (act)
2. Choose one method and give it a variable name (ipcc)
3. `lca = bc.LCA({act: 1}, ipcc[0])` #act:1 is the FU
4. `lca.lci()` # Builds matrices, solves the system
5. `lca.lcia()` # Characterization, i.e. the multiplication of the elements of the LCI matrix with characterization factors from the chosen method
6. `lca.score` #Returns the LCIA score

> Calculating LCIA results for multiple activities (functional units)/methods

Option 1:

1. `acts = list of activities; methods = list of methods`
2. `demands = {f"{a['name']} | {a['location']}": {a.id: 1} for a in acts}`
3. `config = {'impact_categories': methods}`
4. `dobjs = bd.get_multilca_data_objs(functional_units=demands, method_config=config)`
5. `mLCA = bc.MultiLCA(demands=demands, method_config=config, data_objs=dobjs)`
6. `mLCA.lci()` ; `mLCA.lcia()`
7. `mLCA.scores` #dict {(method, FU label): score}
8. `df = pd.Series(mLCA.scores).unstack(level=0)`
9. `df.to_excel('excelname.xlsx')`

Option 2:

```
lca = bc.LCA({first_act: 1}, methods[0])
lca.lci(factorize=True)
lca.lcia()
for act in acts:
    lca.redo_lci({act.id: 1})
    for m in methods:
        lca.switch_method(m)
        lca.lcia()
        print(act["name"], m[1], lca.score)
```

> Plotting results – using pandas dataframes

1. Create a results df as shown above
2. More human friendly labels: `labels_methods = {('EF v3.1', 'climate change', 'global warming potential (GWP100)'): 'CC' \nbio', addmoreasyouneed, }`

```
labels_act = { "'electricity production, wind, >3MW turbine, onshore' (kilowatt hour, DE, None)": "wind", addmoreasyouneed, }
df = df.rename(columns = labels_act, index = labels_methods)
```

3. Plotting

```
df.plot.bar(xlabel='Impact category',
ylabel='Impact score', figsize=(14,8))
plt.xticks(rotation=0)
```

#Normalisation:

```
df = (df.T/df.abs().max(axis=1)).T
```

You can also use seaborn, matplotlib, plotly etc.

> Contribution analysis (Option I)

→ This has become much more intuitive in bw2.5, but polyviz is also very helpful: from `polyviz.utils` import `calculate_supply_chain`

1. Use the «acts» and «methods» list from above

2. `df = pd.DataFrame()`

3. for act in acts:
 for m in methods:
 print(act, m)
 data = calculate_supply_chain(act, m, 2, 1e-4)[0]

#choose down to which level you want to go, and the cutoff

```
data = [[act['name'], act['location'],
'-' .join(m)] + d for d in data]
df = pd.concat([df,
pd.DataFrame(data)], axis=0, ignore_index=True)
df.columns = ['activity', 'location_activity',
'lcia_method', 'level', 'contribution share',
'absolute score', 'amount_in_dataset',
'activity_contributing', 'location_contributing',
'unit']
df.to_excel('contribution.xlsx')
```

> Contribution analysis (more options)

a) Quickest look:

```
import bw2analyzer as ba
ba.print_recursive_calculation(act, method,
max_level=3, cutoff=0.05)
```

b) Ranked tables — top processes and top flows:

```
lca = bc.LCA({act: 1}, method); lca.lci();
lca.lcia()
ca = ba.ContributionAnalysis()
ca.annotated_top_processes(lca, limit=10)  #
(score, supply, activity)
ca.annotated_top_emissions(lca, limit=10)  #
(score, supply, biosphere flow)
```

c) straight to pandas:

```
df = lca.to_dataframe(cutoff=25)  #
characterized inventory, tidy
df[['row_name', 'row_categories', 'col_name', 'col_lo
cation', 'amount']]
```

> Project management: Reproducibility & collaboration

E.g. when you want to freeze the status of a project (e.g. after manuscript submission); or when you want to collaborate: Share env and project with all databases

Saving your env: `conda env export -n envname > C:\yourpath\envname_20240401.yml`

Recreating the env: `conda env create -f C:\yourpath\envname_20240401.yml`

Backup existing project:

```
bi.backup.backup_project_directory(project
='yourname') (saved in your C:\Users folder)
```

Restore project from backup:

```
bi.backup.restore_project_directory(fp
=yourpath, project_name = 'yourprojectname')
```

> Storage space on disk hygiene

bw eats space fast, in this order:

1. Brightway projects C:\Users\<you>\AppData\Local\pylca\Brightway3
2. conda envs + pkgs ...\\miniforge3\envs and ...\\miniforge3\pkgs
3. ecoinvent downloads ...\\AppData\Local\pylca\EcoinventInterface\cache
4. project backups (.tar.gz) wherever you wrote them

Clean up:

1. In a jupyter notebook: `bd.projects.delete_project('projectname', delete_dir=True)`
`bd.projects.purge_deleted_directories()` #orphan dirs of deleted projects

2. In your conda prompt:

```
conda env list          #which envs exist
conda clean --all --dry-run  #how much conda could free
conda clean --all          # safe: never breaks an existing env - only drops caches
conda env remove -n oldenv
```

3. ecoinvent cache: delete whole version folders you no longer import

Rules of thumb

- One env per course/project, delete when done. Export first:
`conda env export -n bw > bw_2026.yml`
- Back up (bi.backup_project_directory) then delete, don't keep both